

Informe Técnico: Diseño e Implementación de Microservicio Consumidor de SOAP API

Raquel Osorio Mamani

1. Análisis del Proyecto Integrador (15 pts)

Esta sección detalla el análisis realizado sobre el endpoint SOAP proporcionado por el proyecto integrador, que es el punto de partida para el diseño y la implementación del microservicio.

- **1.1. Identificación Clara del Endpoint (5 pts)**

El endpoint SOAP específico proporcionado por el "proyecto integrador" y que será consumido por el microservicio se asume localizado en la siguiente:

- Filtrar listas de palabras según un criterio de búsqueda proporcionado.
 1. Entregar una lista de las palabras más populares. Estas funcionalidades son directamente relevantes para el objetivo general del microservicio a desarrollar.

- **1.2. Justificación de la Elección del Endpoint (5 pts)**

La elección del endpoint `http://localhost:3000/api/filtrado` se justifica primordialmente por su alineación directa con los requisitos funcionales del microservicio. El objetivo es interactuar con las capacidades de filtrado de palabras y obtención de palabras populares ofrecidas por el proyecto integrador, y este endpoint específico expone precisamente dichas operaciones.

El WSDL asociado a este endpoint describe las funciones que el servicio web ofrece, permitiendo a un programa cliente (en este caso, nuestro microservicio) determinar qué operaciones están disponibles y cómo invocarlas.³ Al consumir este endpoint, el microservicio puede cumplir su propósito central de actuar como intermediario o consumidor de estas funcionalidades específicas.

Es fundamental reconocer que la elección de consumir un endpoint SOAP/WSDL no es una preferencia arquitectónica en este contexto, sino una restricción impuesta por la naturaleza del "proyecto integrador". El microservicio *debe* interactuar con la interfaz SOAP existente, haciendo que este endpoint sea la única opción viable para lograr la integración requerida.

- **1.3. Descripción Técnica del Endpoint (estructura, método, formato de datos) (5 pts)**

Una comprensión detallada de la estructura técnica del endpoint es crucial para una implementación exitosa del cliente SOAP en el microservicio. Esta descripción se basa en el análisis del archivo WSDL y el protocolo SOAP subyacente.

- **Estructura WSDL:** El archivo WSDL define la interfaz del servicio web en formato XML. Sus elementos clave incluyen ¹:
 - `<definitions>`: Elemento raíz que contiene toda la descripción y declara los namespaces.

- **<types>**: Define las estructuras de datos (tipos complejos y simples) utilizadas por el servicio, generalmente mediante XML Schema Definition (XSD).³ Aquí se definirían las estructuras para la entrada de filtro y las listas de palabras de salida.
- **<message>**: Define los mensajes abstractos intercambiados. Cada mensaje consta de partes (<part>) que representan los parámetros de entrada o salida de una operación, vinculados a los tipos definidos en <types>.¹ Por ejemplo, un mensaje filterWordsRequest podría tener una parte searchCriteria de tipo xsd:string, y un mensaje filterWordsResponse podría tener una parte filteredWords de tipo ListOfStrings (definido en <types>).
- **<portType>**: Define el conjunto abstracto de operaciones que el servicio soporta. Cada <operation> dentro del <portType> especifica los mensajes de entrada, salida y error asociados.¹ Para este endpoint, esperaríamos operaciones como filterWords y getPopularWords.
- **<binding>**: Especifica los detalles concretos del protocolo y formato de datos para un <portType> particular. En este caso, definiría cómo las operaciones abstractas se mapean a mensajes SOAP y se transportan sobre HTTP.¹
- **<service>**: Agrupa un conjunto de puertos relacionados. Contiene el elemento <port>, que asocia un <binding> específico con una dirección de red concreta (la URL del endpoint donde el servicio está escuchando).¹

Una característica importante de WSDL es la separación entre la descripción abstracta (tipos, mensajes, portType) y la descripción concreta (binding, service), lo que permite cierta flexibilidad, aunque la implementación práctica suele estar fuertemente ligada.¹

- **Operaciones SOAP (Métodos)**: Dentro del <portType>, se definirían las operaciones específicas. Basado en la descripción, se esperan al menos dos operaciones:
 - filterWords: Probablemente una operación de tipo **Request-Response**³, donde el cliente envía un mensaje de entrada (input message) con los criterios de filtro y recibe un mensaje de salida (output message) con la lista de palabras filtradas.
 - getPopularWords: También probablemente una operación **Request-Response**, donde el cliente envía una solicitud (quizás sin parámetros específicos) y recibe una lista de palabras populares. Cada operación tendría asociados los <message> correspondientes definidos previamente.
- **Formato de Datos (Mensaje SOAP)**: La comunicación real se realiza mediante mensajes SOAP, que son documentos XML estructurados.⁶ La estructura básica de un mensaje SOAP es⁸:
 - **<soap:Envelope>**: Elemento raíz que encapsula todo el mensaje. Contiene obligatoriamente un elemento <soap:Body> y opcionalmente un <soap:Header>.
 - **<soap:Header>**: Opcional. Puede contener metadatos o información contextual para nodos SOAP intermedios o para el endpoint final (e.g., información de autenticación, ID de transacción).⁶

- **<soap:Body>**: Obligatorio. Contiene la carga útil del mensaje destinada al receptor final.
 - **Solicitud**: Típicamente contiene un elemento XML cuyo nombre coincide con la operación WSDL que se invoca (e.g., `<tns:filterWords>`), y dentro de él, los parámetros definidos en el mensaje de entrada del WSDL (e.g., `<searchCriteria>...</searchCriteria>`).¹⁰
 - **Respuesta**: Contiene un elemento XML que representa la respuesta de la operación (e.g., `<tns:filterWordsResponse>`), y dentro de él, los datos de retorno definidos en el mensaje de salida del WSDL (e.g., `<filteredWords>...</filteredWords>`).¹¹
 - **<soap:Fault>**: Un subelemento especial dentro del `<soap:Body>` que se utiliza exclusivamente para reportar errores ocurridos durante el procesamiento del mensaje en el servidor.⁸

El formato exacto de los datos dentro del `<soap:Body>` está definido por los esquemas (XSD) referenciados o incluidos en la sección `<types>` del WSDL.³

- **Mecanismo de Filtrado**: La especificación del filtro (la palabra o criterio a buscar) se incluiría como un parámetro dentro del elemento de la operación en el `<soap:Body>` de la solicitud SOAP. Por ejemplo:

XML

```
<soap:Body>
  <tns:filterWords xmlns:tns="http://integrator.example.com/WordServices">
    <searchCriteria>ejemplo</searchCriteria>
  </tns:filterWords>
</soap:Body>
```

La forma exacta dependerá de cómo se haya definido la operación y los tipos de datos en el WSDL. Podrían usarse tipos simples como `xsd:string` o tipos complejos si se requieren filtros más estructurados.¹²

El WSDL actúa como un contrato estricto y detallado.² Si bien esto garantiza claridad sobre la estructura y las operaciones, también implica que cualquier cambio significativo en el WSDL del proyecto integrador requerirá una regeneración y posible modificación del código cliente en el microservicio.¹⁴ Esta dependencia inherente del contrato WSDL es una característica clave de la integración basada en SOAP.

2. Diseño del Microservicio (15 pts)

Esta sección aborda el diseño conceptual del microservicio, definiendo su propósito, justificando las tecnologías de comunicación y visualizando el flujo de integración.

- **2.1. Objetivo del Microservicio Claramente Definido (5 pts)**

El objetivo principal de este microservicio es actuar como una **fachada (Facade)** o **intermediario** entre los consumidores internos (otros microservicios, aplicaciones frontend) y el endpoint SOAP externo del proyecto integrador. Su función primordial es consumir las operaciones `filterWords` y `getPopularWords` expuestas por el servicio SOAP y presentar esta funcionalidad de una manera más accesible y moderna, preferiblemente a través de una API REST.

La **propuesta de valor** de este microservicio reside en:

- **Simplificación:** Abstrae la complejidad inherente a la interacción con un servicio SOAP (manejo de WSDL, construcción de mensajes XML, parsing de respuestas SOAP) para los consumidores internos, que pueden preferir interactuar con interfaces RESTful basadas en JSON.¹⁵
- **Encapsulación:** Aísla la dependencia del servicio SOAP externo. Cambios en la implementación interna del servicio SOAP (siempre que no rompan el contrato WSDL) no deberían afectar a los consumidores del microservicio.
- **Adaptación/Transformación:** Proporciona un punto centralizado para transformar los datos entre el formato XML/SOAP del servicio externo y el formato JSON (u otro) deseado por los consumidores internos.
- **Potencial de Optimización:** Puede incorporar funcionalidades adicionales como caché de respuestas (para reducir llamadas al servicio externo), agregación de datos si fuera necesario, o métricas de uso específicas.

El diseño se adhiere a principios clave de la arquitectura de microservicios:

- **Principio de Responsabilidad Única (Single Responsibility Principle - SRP):** El microservicio se centra exclusivamente en la tarea de interactuar con el servicio SOAP de palabras y exponer sus datos relevantes.¹⁶ No debe incluir lógica de negocio no relacionada o funcionalidades pertenecientes a otros dominios (como gestión de usuarios general, facturación, etc.).¹⁸ Su única razón para cambiar debería estar ligada a cambios en el servicio SOAP externo o en los requisitos de cómo se exponen sus datos.
- **Acoplamiento Débil (Loose Coupling):** Se busca minimizar las dependencias. El microservicio estará débilmente acoplado a sus *consumidores* a través de su propia API REST bien definida. Estará acoplado al *servicio SOAP externo* a través del contrato WSDL.¹⁶ Este acoplamiento al WSDL es inherente a la integración SOAP y representa un acoplamiento en tiempo de diseño (design-time coupling)¹⁹; cambios en el WSDL pueden requerir cambios en el microservicio. Sin embargo, al actuar como fachada, evita que *múltiples* consumidores internos se acoplen directamente al WSDL.
- **Alta Cohesión (High Cohesion):** Todos los componentes internos del microservicio (el cliente SOAP generado, la lógica de transformación de datos, el controlador REST que expone la API) están estrechamente relacionados y contribuyen directamente a su responsabilidad única de interactuar con el servicio de palabras.¹⁶

La aplicación de estos principios justifica la creación de este microservicio dedicado. En lugar de que cada posible consumidor implemente su propio cliente SOAP y lógica de transformación, el microservicio centraliza esta interacción, promoviendo la mantenibilidad, la reutilización y la evolución independiente de los sistemas.¹⁶ Aísla la complejidad y la potencial volatilidad del servicio SOAP externo.

- **2.2. Elección y Justificación de la Tecnología de Comunicación (5 pts)**

El microservicio opera con dos flujos de comunicación distintos, cada uno con una tecnología dictada por diferentes factores:

- **Comunicación Externa (Microservicio -> Endpoint SOAP del Proyecto Integrador):**
 - **Tecnología Seleccionada:** SOAP sobre HTTP, utilizando el WSDL proporcionado.
 - **Justificación:** Esta elección es **obligatoria**, determinada por la interfaz expuesta por el "proyecto integrador". El servicio externo define su contrato mediante WSDL y espera comunicaciones SOAP.¹ El microservicio debe, por tanto, actuar como un cliente SOAP para poder consumir la funcionalidad requerida.² No hay alternativa tecnológica para esta interacción específica.
- **Comunicación Interna/Expuesta (Consumidores -> Microservicio):**
 - **Tecnología Recomendada:** REST (Representational State Transfer) sobre HTTP/S, utilizando JSON como formato de datos.
 - **Justificación:** Para la API que el propio microservicio expone a sus consumidores (otros servicios internos, frontends), REST es la opción arquitectónica preferida en arquitecturas de microservicios modernas.²⁰ Sus ventajas sobre SOAP en este contexto incluyen ¹⁴:
 - **Simplicidad y Flexibilidad:** REST utiliza operaciones HTTP estándar (GET, POST, PUT, DELETE) sobre URLs de recursos, lo cual es generalmente más intuitivo y fácil de implementar y consumir que la estructura rígida de SOAP.
 - **Formatos de Datos:** REST permite múltiples formatos de datos, siendo JSON el más común por ser ligero, fácil de parsear en la mayoría de lenguajes (especialmente JavaScript) y nativo para aplicaciones web. SOAP está limitado a XML.
 - **Rendimiento:** Los mensajes JSON suelen ser más pequeños y menos verbosos que los mensajes XML de SOAP, resultando en menor consumo de ancho de banda y procesamiento más rápido. Las respuestas REST pueden ser cacheadas por intermediarios HTTP, mejorando la eficiencia.
 - **Escalabilidad:** La naturaleza sin estado (stateless) de REST simplifica el escalado horizontal, ya que cada solicitud se trata de forma independiente. SOAP puede requerir que el servidor mantenga estado, complicando el escalado.
 - **Alineación con la Web:** REST se alinea mejor con los principios y tecnologías de la web.

La siguiente tabla resume las diferencias clave:**Tabla 1: Comparación Arquitectónica SOAP vs. REST**

Característica	SOAP	REST
Protocolo/Estilo	Protocolo con reglas estrictas y estándares (WSDL, WS-*)	Estilo arquitectónico con principios y restricciones (Stateless, Uniform)
Formato de Datos	Principalmente XML (obligatorio)	JSON (común), XML, Texto plano, HTML
Rendimiento	Generalmente más lento (mensajes más grandes, overhead de XML/envelope)	Generalmente más rápido (mensajes más pequeños, cacheable)
Escalabilidad	Más difícil (puede requerir estado en servidor)	Más fácil (sin estado, arquitectura en capas)
Seguridad	Estándares propios (WS-Security), puede añadir overhead	Aprovecha HTTPS directamente, menos overhead
Estado	Puede ser con estado (stateful)	Sin estado (stateless)
Flexibilidad	Rígido, fuertemente tipado	Flexible, menos restrictivo
Caso de Uso Típico	Sistemas legados, integraciones empresariales, transacciones seguras	APIs públicas, servicios web modernos, aplicaciones móviles, microservicios

Fuente: Sintetizado de [14, 15, 20, 21, 22, 23]

Este microservicio actúa, por tanto, como un ****puente de protocolos**** o ****adaptador****. Traduce las solicitudes entrantes (probablemente REST/JSON) en solicitudes SOAP/XML para el servicio externo, y transforma las respuestas SOAP/XML de vuelta en un formato (probablemente JSON) adecuado para sus consumidores. Esta es una función común y valiosa en arquitecturas que necesitan integrar sistemas modernos con servicios legados basados en SOAP.

- **2.3. Diagrama del Flujo de Integración (5 pts)**

Para visualizar claramente el flujo de comunicación y las interacciones entre los componentes, se recomienda un **Diagrama de Secuencia**. Este tipo de diagrama es eficaz para mostrar el orden temporal de los mensajes intercambiados.

Componentes Involucrados:

- Consumer: Representa cualquier cliente que invoca el microservicio (e.g., otro microservicio, una aplicación frontend).
- API Gateway (Opcional): Punto de entrada único que enruta las solicitudes. Mejora la seguridad, el monitoreo y la gestión.²⁴
- WordServiceFacade (Este Microservicio): El microservicio que estamos diseñando.
- SOAPClient (Componente Interno): La parte del microservicio responsable de la comunicación SOAP.
- ExternalSOAPService: El endpoint SOAP del proyecto integrador (Word Filter/Popularity Service).

Flujo de Interacción (Ejemplo: Solicitud de Filtrado): Fragmento de código

```
@startuml
```

```
actor Consumer
```

```
participant APIGateway <<Optional>>
```

```
participant WordServiceFacade
```

```
participant SOAPClient <<Component>>
```

```
participant ExternalSOAPService
```

```
Consumer -> APIGateway : POST /words/filter\n(Request: JSON with criteria)
```

```
APIGateway -> WordServiceFacade : POST /words/filter\n(Request: JSON with criteria)
```

```
WordServiceFacade -> SOAPClient : buildSOAPRequest(criteria)
```

```
SOAPClient -> ExternalSOAPService : SOAP Request (HTTP
```

```
POST)\n<soap:Body><filterWords><searchCriteria>...</searchCriteria></filterWords></\nsoap:Body>
```

```
activate ExternalSOAPService
```

```
ExternalSOAPService --> SOAPClient : SOAP Response (HTTP 200
```

```
OK)\n<soap:Body><filterWordsResponse><filteredWords>...</filteredWords></filterWo\nrdsResponse></soap:Body>
```

```
deactivate ExternalSOAPService
```

```
SOAPClient -> WordServiceFacade : parseSOAPResponse(soapResponse)
```

```
WordServiceFacade -> WordServiceFacade : transformData(parsedData)
```

```
WordServiceFacade --> APIGateway : Response: JSON with filtered words
```

```
APIGateway --> Consumer : Response: JSON with filtered words
```

```
alt Error Flow
```

```
ExternalSOAPService --> SOAPClient : SOAP Fault (HTTP 500 or 200\nOK)\n<soap:Body><soap:Fault>...</soap:Fault></soap:Body>
```

```

deactivate ExternalSOAPService
SOAPClient -> WordServiceFacade : throw SOAPFaultException
WordServiceFacade --> APIGateway : Error Response (e.g., HTTP 502 Bad Gateway)
APIGateway --> Consumer : Error Response
end
@enduml

```

(Nota: El diagrama anterior está en formato PlantUML. Se puede generar una imagen usando herramientas compatibles o recrearlo en Lucidchart²⁶, Draw.io, etc.)**Explicación del Flujo:**

1. El Consumer envía una solicitud HTTP POST (asumiendo REST) al endpoint /words/filter del WordServiceFacade, posiblemente a través del API Gateway. La solicitud contiene los criterios de filtro en formato JSON.
2. El WordServiceFacade recibe la solicitud JSON.
3. Invoca a su componente interno SOAPClient para construir el mensaje de solicitud SOAP XML, mapeando los criterios JSON al formato esperado por la operación filterWords del WSDL.⁸
4. El SOAPClient envía la solicitud SOAP mediante HTTP POST a la URL del ExternalSOAPService especificada en el WSDL.¹
5. El ExternalSOAPService procesa la solicitud.
6. El ExternalSOAPService devuelve una respuesta SOAP (XML) que contiene la lista de palabras filtradas o un mensaje de error (<soap:Fault>).⁸
7. El SOAPClient recibe la respuesta SOAP.
8. El SOAPClient parsea la respuesta XML y la entrega (posiblemente como objetos Java generados por JAX-WS) al WordServiceFacade.
9. El WordServiceFacade transforma los datos recibidos (objetos Java/XML) al formato JSON esperado por el Consumer.
10. El WordServiceFacade envía la respuesta JSON final al Consumer, de nuevo, posiblemente a través del API Gateway.
11. En caso de error (SOAP Fault), el error se propaga y se devuelve una respuesta de error apropiada al consumidor.

Este diagrama clarifica visualmente el rol del microservicio como intermediario, los protocolos utilizados en cada tramo (REST/JSON en la entrada/salida, SOAP/XML en la comunicación externa) y la naturaleza síncrona de la llamada al servicio SOAP externo. Cualquier retraso o fallo en el ExternalSOAPService impactará directamente en el tiempo de respuesta del WordServiceFacade¹⁹, a menos que se implementen patrones asíncronos adicionales dentro del facade.

3. Implementación Técnica (40 pts)

Esta sección detalla el proceso de construcción del microservicio, enfocándose en la implementación con Java, Spring Boot y JAX-WS como ejemplo principal, pero mencionando

alternativas en Node.js y Python.

- **3.1. Desarrollo del Microservicio Funcional (15 pts)**

La base del microservicio se establece mediante la configuración del proyecto y la generación del cliente SOAP.

- **Configuración del Proyecto (Java/Spring Boot):**

- Se utiliza **Spring Initializr** (start.spring.io) para generar la estructura inicial del proyecto Maven o Gradle.²⁸
- **Dependencias Esenciales:** Se incluyen las siguientes dependencias clave en el archivo pom.xml (Maven) o build.gradle (Gradle):
 - spring-boot-starter-web: Para habilitar la creación de endpoints REST.
 - spring-boot-starter-web-services: Proporciona el soporte fundamental para la creación y consumo de servicios web SOAP con Spring.²⁸
 - (Opcional) lombok: Para reducir código boilerplate en POJOs y DTOs.
- **Estructura del Proyecto:** Se organiza el código en paquetes lógicos, como:
 - com.example.wordservice.controller: Para los @RestController que exponen la API interna.
 - com.example.wordservice.service: Para la lógica de negocio, incluyendo la orquestación de la llamada SOAP y la transformación de datos.
 - com.example.wordservice.config: Para clases de configuración (e.g., configuración del cliente SOAP).
 - com.example.wordservice.dto: Para los Data Transfer Objects usados en la API REST.
 - com.example.wordservice.client.generated: (Directorio de destino para el código generado por wsimport).

- **Generación del Cliente SOAP (Java - wsimport):**

- **Herramienta:** Se utiliza la herramienta wsimport, parte de JAX-WS (Java API for XML Web Services), que viene incluida en el JDK (aunque puede requerir dependencias adicionales a partir de Java 11).³⁰
- **Función:** wsimport lee el archivo WSDL del servicio SOAP externo y genera automáticamente las clases Java necesarias para interactuar con él.³⁰ Estos artefactos incluyen típicamente:
 - **Service Endpoint Interface (SEI):** Una interfaz Java que mapea las operaciones definidas en el <portType> del WSDL.
 - **Service Class:** Una clase que extiende javax.xml.ws.Service y actúa como fábrica para obtener instancias del puerto (SEI).
 - **Request/Response POJOs:** Clases Java que representan los tipos de datos definidos en la sección <types> del WSDL (basados en XSD), utilizados como parámetros de entrada y salida de los métodos de la SEI.

- **Exception Classes:** Clases Java que mapean los elementos <fault> definidos en el WSDL.
- **Automatización (Maven):** Para integrar la generación en el ciclo de vida del build, se configura el plugin jaxws-maven-plugin en el pom.xml.²⁸

XML

```
<plugin>
  <groupId>com.sun.xml.ws</groupId>
  <artifactId>jaxws-maven-plugin</artifactId>
  <version>3.0.2</version> <executions>
    <execution>
      <goals>
        <goal>wsimport</goal>
      </goals>
    </execution>
  </executions>
  <configuration>
    <wsdlUrls>

    <wsdlUrl>http://integrator.example.com/WordServices/FilterAndPopularity?wsdl</wsdlUrl>
    </wsdlUrls>
    <keep>true</keep>
    <packageName>com.example.wordservice.client.generated</packageName>
    <sourceDestDir>${project.build.directory}/generated-sources/jaxws</sourceDestDir>
    <verbose>true</verbose>
    <extension>true</extension>
  </configuration>
  <dependencies>
    <dependency>
      <groupId>com.sun.xml.ws</groupId>
      <artifactId>jaxws-tools</artifactId>
      <version>3.0.0</version> </dependency>
    </dependencies>
  </plugin>
```

Al ejecutar mvn compile o mvn package, este plugin invocará a wsimport automáticamente.

- **Consideraciones:**
 - **Conflictos:** Si el WSDL contiene nombres que generan conflictos en Java (e.g., palabras reservadas) o si se generan múltiples clientes en el mismo paquete con nombres de clase idénticos, wsimport puede fallar o generar código no

compilable.³² En estos casos, se pueden usar archivos de *binding* JAXB/JAX-WS (pasados con la opción -b) para personalizar la generación de código.³³

- **Versiones Java:** A partir de Java 11, JAX-WS y JAXB ya no forman parte del JDK estándar y deben incluirse como dependencias explícitas (como se muestra en la configuración del plugin y en las dependencias generales del proyecto).³¹

- **Alternativas de Configuración:**

- **Node.js:** Se inicializaría un proyecto con npm init y se instalarían las dependencias express y soap (npm install express soap).³⁴ La estructura incluiría archivos para las rutas (endpoints), servicios (lógica SOAP) y la configuración del servidor Express.
- **Python:** Se crearía un entorno virtual y se instalarían flask (o fastapi), zeep y lxml (pip install flask zeep lxml).³⁶ La estructura contendría el archivo principal de la aplicación Flask/FastAPI, módulos para la lógica del cliente SOAP y posiblemente DTOs (usando Pydantic, por ejemplo).

La utilización de herramientas de generación de clientes como wsimport, node-soap o zeep es fundamental porque abstraen gran parte de la complejidad de la comunicación SOAP/XML.³⁰ Permiten al desarrollador interactuar con el servicio externo utilizando objetos y métodos del lenguaje nativo en lugar de construir y parsear manualmente los mensajes XML. Sin embargo, esta abstracción está intrínsecamente ligada al contrato definido por el WSDL, lo que refuerza el acoplamiento en tiempo de diseño mencionado anteriormente.¹⁴ Un cambio en el WSDL requiere regenerar el cliente.

- **3.2. Consumo Correcto del Endpoint Externo (10 pts)**

Una vez generado el cliente, el siguiente paso es invocar las operaciones del servicio SOAP desde la lógica del microservicio.

- **Uso del Cliente (Java/JAX-WS):**

1. **Instanciar el Servicio:** Se crea una instancia de la clase Service generada por wsimport.

Java

```
// Asumiendo que wsimport generó 'WordServicesService'
```

```
WordServicesService serviceFactory = new WordServicesService();
```

30

2. **Obtener el Puerto (SEI):** Se obtiene la instancia del puerto (la interfaz que define las operaciones) desde la fábrica de servicios.

Java

```
// Asumiendo que la SEI es 'WordServicesPortType'
```

```
WordServicesPortType port = serviceFactory.getWordServicesPortTypePort(); // El nombre del método puede variar
```

30

3. **Configurar Endpoint (Opcional):** Si la URL en el WSDL no es correcta para el entorno actual o necesita ser dinámica, se puede sobrescribir.

Java

```
String actualEndpointUrl =
"http://runtime.integrator.example.com/WordServices/FilterAndPopularity"; // Obtener de
configuración
((BindingProvider)
port).getRequestContext().put(BindingProvider.ENDPOINT_ADDRESS_PROPERTY,
actualEndpointUrl);
// Aquí también se pueden configurar timeouts, cabeceras SOAP, etc.
```

29

4. **Preparar la Solicitud:** Se crean instancias de las clases de solicitud generadas y se establecen sus propiedades.

```
Java
// Asumiendo clases generadas FilterWordsRequest y FilteredWordsResponse
FilterWordsRequest filterRequest = new FilterWordsRequest();
filterRequest.setSearchCriteria("palabra_a_filtrar");
```

5. **Invocar la Operación:** Se llama al método correspondiente en la instancia del puerto, pasando el objeto de solicitud.

```
Java
try {
    FilteredWordsResponse soapResponse = port.filterWords(filterRequest);
    // Procesar soapResponse.getFilteredWords();
} catch (SOAPFaultException faultEx) {
    // Manejar errores específicos de SOAP (Faults)
    System.err.println("SOAP Fault: " + faultEx.getFault().getFaultString());
} catch (WebServiceException wsEx) {
    // Manejar otros errores de servicio web (conexión, timeout, etc.)
    System.err.println("WebService Exception: " + wsEx.getMessage());
}
```

29

○ **Uso del Cliente (Node.js - node-soap):**

1. **Crear Cliente:** Se utiliza soap.createClient() o soap.createClientAsync().

```
JavaScript
const soap = require('soap');
const url = 'http://integrator.example.com/WordServices/FilterAndPopularity?wsdl';

async function callFilterWords(criteria) {
    try {
        const client = await soap.createClientAsync(url);
        const args = { searchCriteria: criteria }; // Mapeo de argumentos
        // El nombre del método (e.g., filterWordsAsync) depende de cómo node-soap lo genere
```

```

    const result = await client.filterWordsAsync(args);
    // result usualmente contiene la respuesta parseada
    return result.filteredWords;
  } catch (error) {
    console.error('SOAP Client Error:', error.Fault? error.Fault : error);
    throw error; // Re-lanzar o manejar
  }
}

```

35

2. **Llamada al Método:** Se invoca la función correspondiente en el objeto client, pasando los argumentos como un objeto JavaScript. Se maneja la respuesta (o error) en el callback o mediante await con la versión Async.³⁵ Es importante notar que el nombre del método en el cliente node-soap puede no coincidir exactamente con el nombre de la operación WSDL, y a veces requiere inspeccionar el objeto client o su descripción (client.describe()).

- **Uso del Cliente (Python - zeep):**

1. **Crear Cliente:** Se instancia Client.

```

Python
from zeep import Client
from zeep.exceptions import Fault

wsdl_url = 'http://integrator.example.com/WordServices/FilterAndPopularity?wsdl'
client = Client(wsdl=wsdl_url)

def call_filter_words(criteria):
    try:
        # El nombre del servicio y la operación dependen del WSDL
        # Se asume 'WordServices' como servicio y 'filterWords' como operación
        response = client.service.filterWords(searchCriteria=criteria)
        return response.filteredWords # Acceso a los datos de respuesta
    except Fault as fault:
        print(f"SOAP Fault: {fault.message}")
        # print(f"Fault Detail: {fault.detail}") # El detalle puede ser XML
        raise fault
    except Exception as e:
        print(f"Error: {e}")
        raise e

```

36

2. **Llamada al Método:** Se accede a la operación a través de

client.service.nombreOperacion()) y se pasan los argumentos (como argumentos nombrados o un diccionario si la estructura es compleja).¹¹ Zeep maneja la serialización a XML y la deserialización de la respuesta.

- **Manejo de Errores:** Es crucial implementar un manejo robusto de errores. Esto incluye capturar Fault específicos de SOAP⁸, que indican errores de negocio o procesamiento en el servidor SOAP, así como excepciones generales de red, timeouts o errores de parseo.

Aunque las bibliotecas cliente simplifican enormemente la interacción, una comprensión de la estructura subyacente del mensaje SOAP⁶ sigue siendo valiosa. En casos de errores inesperados, configuración de cabeceras personalizadas (como WS-Security) o tipos de datos complejos, puede ser necesario inspeccionar los mensajes XML crudos enviados y recibidos⁴¹ utilizando herramientas de debugging o interceptores/handlers proporcionados por la biblioteca cliente.⁴¹

● 3.3. Procesamiento y Transformación de los Datos (10 pts)

Un paso esencial en el microservicio de fachada es la transformación de datos entre la API interna (REST/JSON) y la API externa (SOAP/XML).

- **Mapeo de Entrada (REST -> SOAP):** Cuando el microservicio recibe una solicitud en su endpoint REST (e.g., un JSON en el body de un POST a /words/filter), los datos de esa solicitud (e.g., { "criteria": "buscar" }) deben ser mapeados a los objetos de solicitud Java generados por wsimport (e.g., FilterWordsRequest con su método setSearchCriteria("buscar")). En Node.js o Python, se mapearía el JSON de entrada al objeto JavaScript o diccionario Python esperado por la función del cliente SOAP.³⁵
- **Transformación de Salida (SOAP -> REST):** Inversamente, la respuesta recibida del servicio SOAP (e.g., un objeto FilteredWordsResponse en Java, o el objeto resultado en Node.js/Python) debe ser transformada al formato que la API REST del microservicio promete a sus consumidores. Generalmente, esto implica mapear los datos de los objetos de respuesta SOAP a **Data Transfer Objects (DTOs)** diseñados específicamente para la API REST. Estos DTOs se serializarán luego a JSON.

Java

// Ejemplo de DTO para la respuesta REST

```
public class FilteredWordsDTO {  
    private List<String> words;  
    // Getters y Setters  
}
```

// En la capa de servicio Java:

```
FilteredWordsResponse soapResponse = port.filterWords(filterRequest);
```

```
FilteredWordsDTO dto = new FilteredWordsDTO();
```

```
// Mapear de soapResponse.getFilteredWords() a dto.setWords(...)
```

```
return dto; // Este DTO será serializado a JSON por Spring
```

- **Lógica de Procesamiento Adicional:** Este es el lugar para cualquier lógica adicional requerida:
 - **Validación:** Validar los datos recibidos del cliente REST antes de enviarlos al servicio SOAP.
 - **Combinación:** Si fuera necesario llamar a múltiples operaciones SOAP (e.g., filterWords y luego getPopularWords) para construir una única respuesta REST.
 - **Formateo:** Ajustar formatos de datos si es necesario.
 - **Enriquecimiento:** Añadir información adicional no proveniente del servicio SOAP.
- **Herramientas de Mapeo (Java):** Para mapeos complejos entre los objetos generados por JAX-WS y los DTOs REST, se pueden usar bibliotecas como **ModelMapper** o **MapStruct** para automatizar y simplificar el proceso, reduciendo el código manual de getters y setters. Para casos simples, el mapeo manual puede ser suficiente. En Node.js y Python, el mapeo suele hacerse mediante manipulación directa de objetos y diccionarios. Si se trabaja directamente con XML (menos común con clientes de alto nivel), se usarían bibliotecas como xml2js (Node.js) ⁴³ o lxml (Python).

Esta capa de transformación es vital. Desacopla eficazmente el contrato de la API interna (REST/JSON) del contrato de la API externa (SOAP/WSDL). Si la estructura interna del servicio SOAP cambia pero la información fundamental sigue disponible, idealmente solo esta capa de transformación necesitaría ajustarse, manteniendo estable la API REST expuesta a los consumidores internos. Actúa como un amortiguador contra los cambios en el sistema externo.

● 3.4. Exposición de Endpoint Propio/Documentado (5 pts)

El microservicio debe exponer su propia interfaz para ser consumido por otros servicios o aplicaciones.

- **Diseño de API (REST):** Se define una API REST clara y consistente. Siguiendo el ejemplo:
 - POST /words/filter: Acepta un cuerpo JSON con criterios de búsqueda (e.g., {"criteria": "texto"}) y devuelve una lista de palabras filtradas (e.g., {"words": ["palabra1", "palabra2"]}).
 - GET /words/popular: No requiere cuerpo de solicitud y devuelve una lista de palabras populares (e.g., {"popularWords": ["popular1", "popular2"]}). Se utiliza JSON como formato estándar de solicitud y respuesta.
- **Implementación del Endpoint (Java/Spring Boot):** Se utiliza @RestController para definir la clase controladora y anotaciones como @PostMapping, @GetMapping, @RequestBody para mapear las rutas HTTP y los datos.

Java

```
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
```

```

import java.util.List; // Asumiendo DTOs apropiados

@RestController
@RequestMapping("/words") // Ruta base para las palabras
public class WordController {

    @Autowired
    private WordIntegrationService wordIntegrationService; // Servicio que llama al cliente
    SOAP

    @PostMapping("/filter")
    public ResponseEntity<FilteredWordsDTO> filterWords(@RequestBody FilterCriteriaDTO criteria) {
        try {
            FilteredWordsDTO result = wordIntegrationService.filterWords(criteria);
            return ResponseEntity.ok(result);
        } catch (Exception e) {
            // Manejo de errores adecuado, devolver un estado HTTP apropiado
            return ResponseEntity.status(500).build(); // Ejemplo simplificado
        }
    }

    @GetMapping("/popular")
    public ResponseEntity<PopularWordsDTO> getPopularWords() {
        try {
            PopularWordsDTO result = wordIntegrationService.getPopularWords();
            return ResponseEntity.ok(result);
        } catch (Exception e) {
            return ResponseEntity.status(500).build(); // Ejemplo simplificado
        }
    }
}

```

29

- **Implementación (Node.js/Express):** Se definirían rutas usando `app.post('/words/filter', async (req, res) => {... })` y `app.get('/words/popular', async (req, res) => {... })`, donde los handlers llamarían a la lógica que interactúa con node-soap.³⁵
- **Implementación (Python/Flask):** Se usarían decoradores como `@app.route('/words/filter', methods=)` y `@app.route('/words/popular')` para definir los endpoints.⁴⁴

- **Documentación de la API:** Es fundamental documentar esta API REST expuesta. Se recomienda utilizar la especificación **OpenAPI (anteriormente Swagger)**.
 - **Generación:** En Java/Spring Boot, herramientas como **Springdoc-openapi** pueden generar automáticamente la especificación OpenAPI a partir de las anotaciones del controlador.
 - **Contenido:** La especificación describe los endpoints, métodos HTTP, parámetros de entrada (query, path, body), formatos de solicitud/respuesta (esquemas JSON), y posibles códigos de estado de respuesta.
 - **Interfaz de Usuario:** Herramientas como Swagger UI pueden renderizar la especificación OpenAPI en una interfaz web interactiva, facilitando la exploración y prueba de la API por parte de los consumidores.²⁶

La exposición de una API REST bien definida y documentada es lo que hace que el microservicio sea útil para el ecosistema interno. Oculta la complejidad de la interacción SOAP subyacente y proporciona un contrato claro y moderno para los servicios que necesitan consumir la funcionalidad de filtrado y popularidad de palabras. La documentación OpenAPI actúa como el contrato formal para estos consumidores internos.

● 3.5. Comparación de Herramientas SOAP (Java)

Al implementar un cliente SOAP en Java, existen varias bibliotecas y frameworks. Las opciones más comunes son JAX-WS RI (la implementación de referencia incluida a menudo con el JDK o como dependencia separada) y Apache CXF.

Tabla 2: Comparación de Bibliotecas Cliente SOAP en Java

Característica	JAX-WS RI (wsimport)	Apache CXF (wsdl2java)
Origen	Implementación de referencia del estándar JAX-WS	Proyecto de la Apache Software Foundation
Generación Cliente	Herramienta wsimport ³⁰	Herramienta wsdl2java ²
Estándares WS-*	Soporte básico para estándares JAX-WS. Soporte WS-* limitado/requiere configuración adicional.	Amplio soporte para estándares WS-* (WS-Security, WS-Addressing, WS-ReliableMessaging, etc.)
Integración Spring	Buena integración con spring-boot-starter-web-services. ²⁸	Excelente integración con Spring (módulos cxf-spring-boot-starter-jaxws). Puede ser más flexible.

Flexibilidad	Menos flexible en configuración avanzada y extensiones.	Muy flexible y extensible mediante interceptores.
Comunidad/Soporte	Ampliamente utilizado, parte del estándar Java EE/Jakarta EE.	Comunidad activa de Apache, buena documentación.
Facilidad de Uso	Generalmente más simple para casos de uso básicos.	Puede requerir más configuración inicial para funciones avanzadas.
Dependencias	Requiere dependencias JAX-WS/JAXB explícitas (Java 11+).	Trae sus propias dependencias, gestionadas por Maven/Gradle.

Fuente: Sintetizado de [2, 28, 30, 31, 32]

****Justificación de la Elección (Ejemplo):**** Para este proyecto, se ha optado por utilizar ****JAX-WS RI**** con ``spring-boot-starter-web-services``. Esta elección se basa en:

* ****Simplicidad:**** Para el consumo básico de un endpoint SOAP sin requisitos complejos de WS-*, JAX-WS RI ofrece una configuración y uso relativamente sencillos, especialmente con la integración de Spring Boot.[28, 29]

* ****Estándar:**** Se basa directamente en el estándar JAX-WS.

* ****Suficiencia:**** Cumple con los requisitos del proyecto de consumir el endpoint SOAP especificado.

Si el proyecto requiriera características avanzadas de WS-* (como seguridad compleja o mensajería fiable) o una mayor flexibilidad en la configuración del pipeline de procesamiento de mensajes, ****Apache CXF**** sería una alternativa más potente a considerar.

(Se omiten las comparaciones para Node.js y Python por brevedad, pero seguirían una lógica similar: ``node-soap`` [35] es el estándar de facto en Node.js, y ``zeep`` [36, 37] es la opción moderna y recomendada en Python).

4. Pruebas y Documentación (15 pts)

La verificación del correcto funcionamiento y la documentación clara son esenciales para la

calidad y mantenibilidad del microservicio.

- **4.1. Evidencias Funcionales (capturas, video, Postman, etc.) (5 pts)**

El propósito de las evidencias funcionales es demostrar empíricamente que el microservicio cumple con sus objetivos: consumir correctamente el endpoint SOAP externo y exponer la funcionalidad a través de su propia API REST.

Métodos de Recopilación:

- **Postman/Herramientas Similares:** Es la herramienta principal para probar la API REST expuesta por el microservicio.
 - **Capturas de Pantalla:** Incluir capturas de pantalla de Postman ⁴⁵ mostrando:
 - La configuración de la solicitud (URL del endpoint REST del microservicio, método POST/GET, cabeceras como Content-Type: application/json, cuerpo de la solicitud JSON para POST /words/filter).
 - La respuesta recibida (código de estado 200 OK, cabeceras, cuerpo de la respuesta JSON con los datos esperados - palabras filtradas o populares).
 - **Colecciones Postman:** Exportar la colección de Postman utilizada para las pruebas puede servir como evidencia ejecutable.
- **SoapUI (para el endpoint SOAP externo):** Aunque no es una prueba directa del *microservicio*, puede ser útil incluir capturas de SoapUI ⁴⁸ interactuando directamente con el endpoint SOAP del proyecto integrador. Esto sirve para:
 - Verificar la estructura esperada de solicitud/respuesta SOAP.
 - Comparar la respuesta directa del SOAP con la respuesta transformada del microservicio.
 - Validar el comportamiento del servicio externo independientemente del microservicio.
- **Logs del Microservicio:** Incluir fragmentos relevantes de los logs generados por el microservicio durante una solicitud exitosa. Estos logs deberían mostrar:
 - Recepción de la solicitud REST.
 - Construcción de la solicitud SOAP.
 - (Opcional, si se loguea) Envío de la solicitud SOAP al endpoint externo.
 - (Opcional, si se loguea) Recepción de la respuesta SOAP.
 - Transformación de los datos.
 - Envío de la respuesta REST final.
- **Video Corto:** Una grabación de pantalla mostrando el proceso completo: enviar una solicitud con Postman al microservicio, observar (quizás a través de logs o un debugger) la llamada SOAP subyacente, y ver la respuesta correcta en Postman.
- **Datos de Ejemplo:** Proporcionar ejemplos concretos de datos de entrada (JSON para REST) y los datos de salida esperados (JSON de REST) para los escenarios probados.

Estas evidencias funcionales son cruciales porque demuestran el comportamiento real del

sistema integrado. Confirman que la comunicación entre el consumidor, el microservicio y el servicio SOAP externo funciona como se diseñó en el diagrama de flujo (Sección 2.3). Las herramientas como Postman y SoapUI permiten realizar estas pruebas de forma estructurada y repetible.⁴⁵

- **4.2. Pruebas Unitarias o Manuales Explicadas (5 pts)**

Se debe implementar una estrategia de pruebas que cubra diferentes niveles de la aplicación.

- **Pruebas Unitarias:**

- **Objetivo:** Verificar el correcto funcionamiento de las unidades de código más pequeñas (métodos individuales, clases) de forma aislada del resto del sistema y, fundamentalmente, **sin dependencias externas** como el servicio SOAP real o bases de datos.
- **Componentes a Probar:**
 - **WordController (Controlador REST):** Verificar que mapea correctamente las solicitudes HTTP, deserializa el JSON de entrada, llama al método apropiado del servicio y serializa la respuesta a JSON. Se usa MockMvc en Spring Boot.
 - **WordIntegrationService (Capa de Servicio):** Verificar la lógica de orquestación, la llamada (mockeada) al cliente SOAP y la lógica de transformación de datos.
 - **Lógica de Transformación/Mapeo:** Probar específicamente las funciones o clases responsables de convertir entre DTOs REST y objetos SOAP.
- **Mocking de Dependencias:** Es **esencial** mockear (simular) la interacción con el cliente SOAP. Las pruebas unitarias **no deben** realizar llamadas de red reales al servicio SOAP externo.
 - **Java (JUnit/Mockito):** Se utiliza @Mock para crear un doble de prueba (mock) del WordServicesPortType (la SEI generada) o de una clase wrapper que encapsule al cliente SOAP. Se utiliza @InjectMocks en la clase bajo prueba (e.g., WordIntegrationService) para inyectar el mock.⁵¹ Con when(mockPort.filterWords(...)).thenReturn(mockSoapResponse) se define el comportamiento del mock para simular una respuesta SOAP exitosa.⁴² Con when(...).thenThrow(new SOAPFaultException(...)) se simulan errores SOAP (Faults).⁴²

```
Java
// Ejemplo de Test Unitario para WordIntegrationService
@ExtendWith(MockitoExtension.class)
class WordIntegrationServiceTest {
    @Mock
    private WordServicesPortType mockSoapPort; // Mock del cliente SOAP generado

    @InjectMocks
    private WordIntegrationService serviceUnderTest;
```

```

@Test
void testFilterWords_Success() {
    // Arrange
    FilterCriteriaDTO criteria = new FilterCriteriaDTO("test");
    FilteredWordsResponse mockSoapResponse = new
FilteredWordsResponse();
    //... poblar mockSoapResponse con datos simulados...

when(mockSoapPort.filterWords(any(FilterWordsRequest.class))).thenReturn(
mockSoapResponse);

    // Act
    FilteredWordsDTO result = serviceUnderTest.filterWords(criteria);

    // Assert
    //... verificar que result contiene los datos transformados esperados...
    verify(mockSoapPort, times(1)).filterWords(any(FilterWordsRequest.class));
// Verificar llamada al mock
}

@Test
void testFilterWords_SoapFault() {
    // Arrange
    FilterCriteriaDTO criteria = new FilterCriteriaDTO("error");
    SOAPFault fault = mock(SOAPFault.class); // Mockear el fault si es necesario
    when(fault.getFaultString()).thenReturn("Invalid criteria");

when(mockSoapPort.filterWords(any(FilterWordsRequest.class))).thenThrow(new
SOAPFaultException(fault));

    // Act & Assert
    assertThrows(ExternalServiceException.class, () -> { // Asumiendo una
excepción personalizada
        serviceUnderTest.filterWords(criteria);
    });
    verify(mockSoapPort,
times(1)).filterWords(any(FilterWordsRequest.class));
}
}

```

- **Node.js (Jest/Mocha):** Se utilizan funciones mock de Jest (jest.fn()) o bibliotecas como sinon para reemplazar los métodos del cliente node-soap y controlar sus respuestas/errores.
- **Python (Pytest/unittest.mock):** Se usa el decorador @patch o la fixture mocker de pytest-mock para reemplazar los métodos del servicio zeep

(client.service.filterWords) y definir sus valores de retorno o excepciones.⁵²

- **Pruebas de Integración:**

- **Objetivo:** Verificar la correcta interacción entre el microservicio y el componente externo con el que se integra, en este caso, el servicio SOAP. Se prueba el flujo completo desde la recepción de la solicitud REST hasta la recepción y procesamiento de la respuesta SOAP (real o mockeada a nivel HTTP).

- **Estrategias:**

1. **Mocking a Nivel HTTP/Transporte:** Esta es la estrategia preferida para pruebas de integración automatizadas y fiables. Se utilizan herramientas que interceptan las llamadas HTTP salientes destinadas al endpoint SOAP real y devuelven respuestas XML predefinidas.

- **Java:** Se puede usar mockito-soap-cxf⁴² (si se usa CXF o para crear un endpoint mock local) o bibliotecas como WireMock o Hoverfly que actúan como servidores HTTP mock.

- **Node.js: Nock** es una biblioteca popular para interceptar solicitudes HTTP salientes y devolver respuestas mockeadas. Permite mockear tanto la solicitud GET para el WSDL como la solicitud POST para la operación SOAP, verificando cabeceras como SOAPAction.⁵⁴

- **Python: requests-mock** se integra bien con zeep (que usa requests por debajo) para mockear las respuestas HTTP a las llamadas SOAP.⁵³ Estas pruebas validan que el cliente SOAP generado (e.g., por wsimport) puede construir correctamente la solicitud XML y parsear correctamente la respuesta XML mockeada, sin depender de la disponibilidad o estado del servicio SOAP real.

2. **Pruebas contra Servicio Real/Contenedorizado:** Se puede configurar un entorno de prueba donde se despliega el servicio SOAP real (si es controlable y estable) o una versión mock completa del servicio SOAP (e.g., usando SoapUI MockService o un servicio mock dedicado) en un contenedor Docker.⁵⁶ Las pruebas de integración se ejecutan entonces contra esta instancia. Esto ofrece la máxima fidelidad pero incrementa la complejidad de la configuración del entorno de pruebas y puede ser más lento y menos fiable.

- **Pruebas Manuales:** Si la cobertura de pruebas automatizadas no es completa, describir los pasos manuales realizados para verificar la funcionalidad. Esto típicamente implica usar Postman o SoapUI (como en la Sección 4.1) para probar escenarios específicos (casos borde, entradas inválidas) y verificar visualmente las respuestas.

La combinación de pruebas unitarias (rápidas, enfocadas en la lógica interna, con dependencias mockeadas⁵¹) y pruebas de integración (verificando la correcta interacción con el contrato externo, usando mocks a nivel HTTP⁵³ o instancias reales/mockeadas⁵⁶) proporciona una estrategia de pruebas robusta y equilibrada.

- **4.3. Documentación Clara del Microservicio (README.md o PDF) (5 pts)**

La documentación es fundamental para que otros desarrolladores (o el propio desarrollador en el futuro) puedan entender, utilizar, construir y mantener el microservicio. Se requieren dos niveles de documentación: el README.md enfocado en el código y un documento técnico más detallado (PDF).

- **Buenas Prácticas para README.md:**

- **Propósito:** Es el punto de entrada principal para cualquier persona que interactúe con el repositorio de código fuente.²⁶ Debe proporcionar información esencial para ponerse en marcha rápidamente.
- **Estructura Esencial:** Un README.md bien estructurado para un microservicio debería incluir las siguientes secciones ²⁵:

Tabla 3: Secciones Esenciales para README.md de Microservicio

Sección	Descripción	Ejemplo de Contenido
Título del Proyecto	Nombre claro y descriptivo del microservicio.	Word Service Facade Microservice
Descripción Breve	Un párrafo resumiendo el propósito y la funcionalidad principal del microservicio.	"Microservicio que actúa como fachada para el servicio SOAP externo de filtrado y popularidad de palabras."
Stack Tecnológico	Lista de las principales tecnologías, lenguajes y frameworks utilizados.	"Java 17, Spring Boot 3.x, Maven, JAX-WS, Lombok, JUnit 5, Mockito, Docker."
Prerrequisitos	Software necesario para construir y ejecutar el proyecto localmente.	"JDK 17+, Maven 3.8+, Docker (opcional, para ejecución en contenedor)."
Puesta en Marcha	Instrucciones paso a paso para clonar el repositorio, instalar dependencias y construir el proyecto.	git clone..., cd..., mvn clean package
Ejecución Local	Cómo ejecutar el microservicio en un entorno de desarrollo.	mvn spring-boot:run o java -jar target/word-service-facade-0.0.1-SNAPSHOT.jar
Ejecución de Pruebas	Comando para ejecutar las pruebas unitarias y/o de	mvn test

	integración.	
Endpoints de la API	Resumen de los endpoints REST expuestos por el microservicio. Enlace a la documentación OpenAPI/Swagger.	- POST <code>http://localhost:8001/wsdl</code>
Configuración	Descripción de las variables de entorno o archivos de configuración necesarios (e.g., URL del SOAP).	SOAP_ENDPOINT_URL=http://..., SERVER_PORT=8080
Despliegue (Opcional)	Breves notas sobre cómo desplegar el servicio (e.g., comandos Docker, scripts de despliegue).	<code>docker build -t word-service.</code> , <code>docker run -p 8080:8080...</code>
Contribución (Opcional)	Guía para contribuir al proyecto.	"Seguir el estilo de código X, crear Pull Requests..."
Licencia	Información sobre la licencia del software.	"Licencia MIT"

Fuente: Sintetizado de [25, 27, 58, 59] y buenas prácticas generales.

* **Claridad y Concisión:** Utilizar lenguaje claro, formato Markdown adecuado (listas, bloques de código), y mantener las instrucciones concisas y fáciles de seguir.

* **Documentación Técnica Detallada (PDF):** Se debe indicar en el README que la documentación técnica exhaustiva (que cubre arquitectura, decisiones de diseño, análisis del WSDL, flujo detallado, etc.) se encuentra en un documento PDF separado, como se especifica en el criterio 5.1. El README se centra en el **uso y desarrollo** del código base.

Un `README.md` bien elaborado es una inversión crucial en la usabilidad y mantenibilidad del microservicio.[26] Reduce significativamente la barrera de entrada para nuevos desarrolladores o para cualquiera que necesite interactuar con el código, construirlo, probarlo o entender su configuración básica.[25, 59]

5. Presentación Final (15 pts)

Esta sección describe los componentes finales de la entrega, enfocándose en la documentación técnica profunda y la demostración funcional.

- **5.1. Explicación Técnica del Microservicio y su Integración en PDF Adjunto al Repositorio (10 pts)**

Este documento PDF constituye la explicación técnica detallada y exhaustiva del microservicio, su diseño, implementación e integración. Su propósito es ir más allá del README.md, proporcionando un contexto arquitectónico y de diseño completo para evaluadores o para la base de conocimiento del equipo.²⁶

Estructura y Contenido Recomendados:⁶⁰

1. **Introducción:**
 - Objetivo general del proyecto.
 - Propósito específico y valor del microservicio WordServiceFacade.
2. **Visión General de la Arquitectura:**
 - Diagrama de alto nivel mostrando el microservicio en su ecosistema (consumidores, API Gateway si existe, servicio SOAP externo).²⁶
 - Breve descripción de los componentes principales y sus interacciones.
3. **Decisiones de Diseño:**
 - Justificación de la arquitectura de microservicios.
 - Aplicación de principios: SRP, Acoplamiento Débil, Alta Cohesión.¹⁶
 - Justificación de REST/JSON para la API expuesta vs. SOAP/XML para la externa.¹⁵
 - Mención del patrón Facade/Adapter.
4. **Análisis del Endpoint SOAP Externo:**
 - Resumen del análisis del WSDL (elementos clave: types, messages, portType, binding, service).¹
 - Operaciones consumidas (filterWords, getPopularWords).
 - Estructura esperada de los mensajes SOAP (Envelope, Header, Body, Fault).⁸
 - Formato de datos XML esperado (referencia a XSD).
5. **Flujo de Integración Detallado:**
 - Inclusión y explicación detallada del Diagrama de Secuencia (o similar) de la Sección 2.3.²⁴
 - Descripción paso a paso del flujo de una solicitud, desde el consumidor hasta el servicio SOAP y de vuelta.
6. **Detalles de Implementación:**
 - Stack tecnológico utilizado (Java, Spring Boot, JAX-WS, etc.).⁶⁰

- Proceso de generación del cliente SOAP (wsimport y configuración del plugin Maven).³¹
- Fragmentos de código clave comentados:
 - Instanciación y llamada al cliente SOAP.²⁹
 - Lógica de transformación de datos (SOAP <-> DTOs REST).
 - Implementación del @RestController.²⁹
- 7. **Gestión de Datos:**
 - Cómo se reciben, procesan y transforman los datos.
 - Estrategias de caché (si se implementaron, e.g., con Redis ⁶⁰).
- 8. **Especificación de la API Expuesta (REST):**
 - Descripción detallada de cada endpoint REST (/words/filter, /words/popular).
 - Métodos HTTP, parámetros, formato de solicitud (JSON), formato de respuesta (JSON), códigos de estado.
 - Se puede incluir la especificación OpenAPI (YAML o JSON) directamente o enlazar a ella.²⁶
- 9. **Estrategia de Pruebas:**
 - Resumen de las pruebas unitarias realizadas (enfoque, mocking).⁴²
 - Resumen de las pruebas de integración (enfoque, mocking HTTP o contra instancia).⁵³
 - Referencia a las evidencias funcionales.
- 10. **Despliegue:**
 - Descripción de la contenedorización (referencia al Dockerfile).⁵⁹
 - Ejemplo de manifiestos de despliegue (e.g., Kubernetes YAML para Deployment y Service).⁶⁰
- 11. **Conclusión:**
 - Resumen de los logros del proyecto.
 - Posibles mejoras o trabajo futuro.⁶⁰
- **5.2. Demostración Funcional Clara (5 pts)**

La demostración funcional es la validación práctica y visible de que el microservicio opera según lo esperado. Debe presentarse de manera clara y concisa, típicamente referenciada o incluida como parte de la presentación final (posiblemente dentro del PDF o como un enlace a un video).

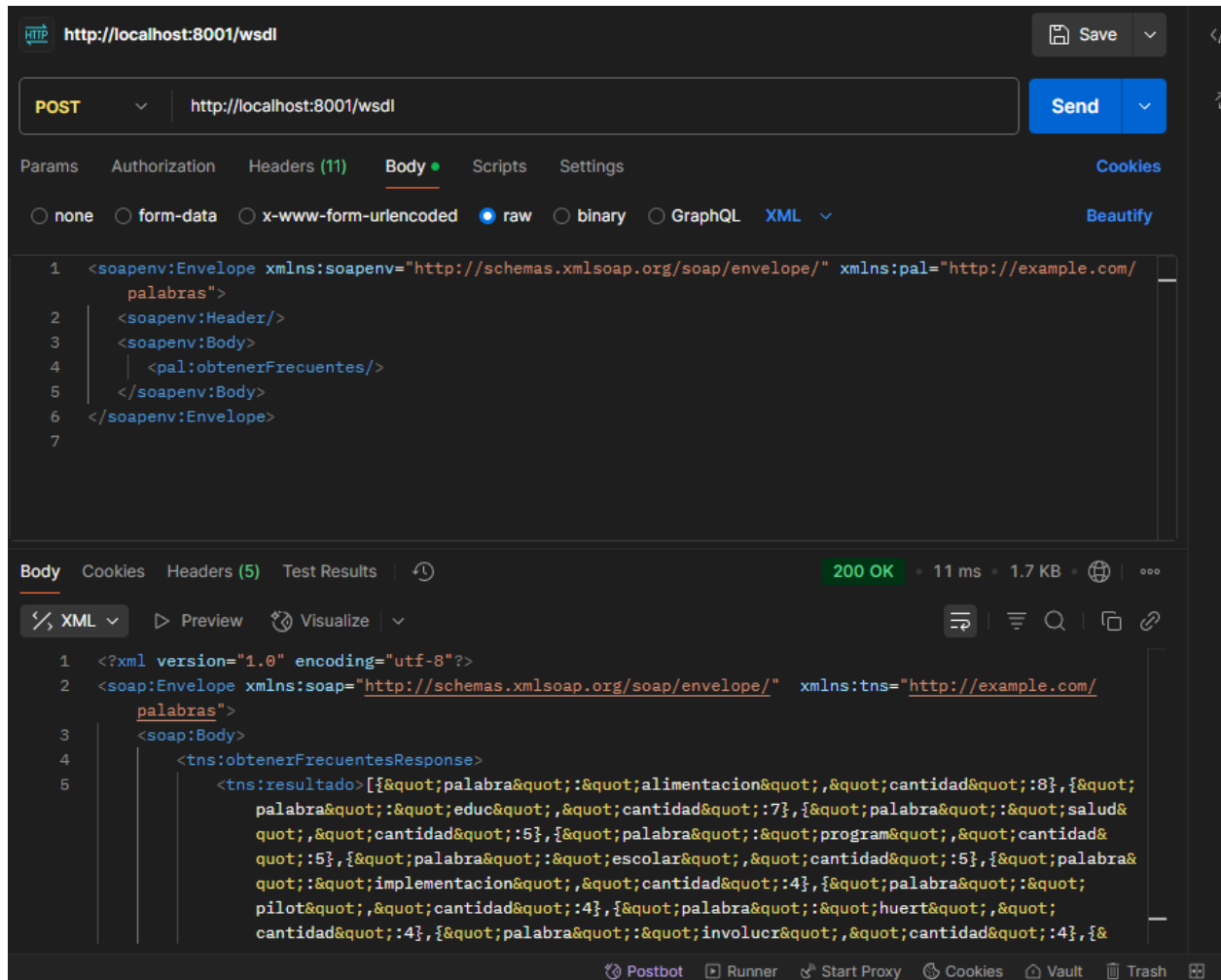
Enfoque de la Demostración:

- **Herramienta:** Utilizar una herramienta como Postman para interactuar con la API REST expuesta por el microservicio.⁴⁵
- **Pasos Claros:** Mostrar una secuencia lógica de acciones:
 1. **Configurar la Solicitud:** Mostrar la configuración de una solicitud en Postman dirigida al endpoint REST del microservicio (e.g., POST `http://localhost:8080/words/filter`). Incluir el método, la URL, las cabeceras

(Content-Type: application/json) y el cuerpo de la solicitud (JSON con criterios de filtro).

2. **Enviar la Solicitud:** Ejecutar la solicitud desde Postman.
3. **Observar el Procesamiento (Opcional pero recomendado):** Si es posible, mostrar brevemente los logs del microservicio en ejecución para evidenciar que la solicitud fue recibida y que se está realizando la llamada SOAP subyacente.
4. **Verificar la Respuesta:** Mostrar la respuesta recibida en Postman. Verificar que el código de estado sea el esperado (e.g., 200 OK) y que el cuerpo de la respuesta (JSON) contenga los datos correctos y transformados (la lista de palabras filtradas).
5. **Repetir (si aplica):** Mostrar también una llamada al endpoint GET /words/popular y verificar su respuesta.

Relación con Evidencias: La demostración debe ser una presentación en vivo (o grabada) de las evidencias funcionales recopiladas en la Sección 4.1. Las capturas de pantalla o el video de esa sección pueden usarse directamente aquí o servir como base para la demostración. **Claridad:** La clave es la claridad. La demostración debe ser fácil de seguir, incluso para alguien que no esté profundamente familiarizado con los detalles internos. Debe confirmar visualmente que el flujo de integración diseñado (Sección 2.3) se ejecuta correctamente y produce el resultado esperado. Esta demostración funcional cierra el ciclo, conectando el diseño teórico y la implementación del código con un resultado tangible y verificable. Es la prueba final de que el microservicio cumple su objetivo principal.



```
// controllers/palabrasSoapController.js
```

```
import axios from 'axios';
```

```
const BACK_API_URL = 'http://localhost:3000/api/filtrado'; // Asegurate que esté corriendo tu back
```

```
export const palabrasService = {
```

```
  PalabrasService: {
```

```
    PalabrasPort: {
```

```
      async obtenerFrecuentes(_, callback) {
```

```
        try {
```

```
          const res = await axios.get(`${BACK_API_URL}`);
```

```
          callback(null, { resultado: JSON.stringify(res.data.top_palabras) });
```

```
        } catch (error) {
```

```

        callback({ fault: { faultstring: 'Error al consumir API REST de palabras frecuentes' } });
    },
    async obtenerPopulares(_, callback) {
        try {
            const res = await axios.get(`${BACK_API_URL}/populares`);
            callback(null, { resultado: JSON.stringify(res.data.populares) });
        } catch (error) {
            callback({ fault: { faultstring: 'Error al consumir API REST de palabras populares' } });
        }
    }
}
};

```

// WSDL

```

<definitions name="PalabrasService"
    targetNamespace="http://example.com/palabras"
    xmlns="http://schemas.xmlsoap.org/wsdl/"
    xmlns:tns="http://example.com/palabras"
    xmlns:soap="http://schemas.xmlsoap.org/wsdl/soap/"
    xmlns:xsd="http://www.w3.org/2001/XMLSchema">

    <message name="obtenerFrecuentesRequest"/>
    <message name="obtenerFrecuentesResponse">
        <part name="resultado" type="xsd:string"/>
    </message>

    <message name="obtenerPopularesRequest"/>
    <message name="obtenerPopularesResponse">
        <part name="resultado" type="xsd:string"/>
    </message>

    <portType name="PalabrasPortType">
        <operation name="obtenerFrecuentes">
            <input message="tns:obtenerFrecuentesRequest"/>

```

```

        <output message="tns:obtenerFrecuentesResponse"/>
    </operation>
    <operation name="obtenerPopulares">
        <input message="tns:obtenerPopularesRequest"/>
        <output message="tns:obtenerPopularesResponse"/>
    </operation>
</portType>

<binding name="PalabrasBinding" type="tns:PalabrasPortType">
    <soap:binding style="rpc" transport="http://schemas.xmlsoap.org/soap/http"/>
    <operation name="obtenerFrecuentes">
        <soap:operation soapAction="obtenerFrecuentes"/>
        <input><soap:body use="literal"/></input>
        <output><soap:body use="literal"/></output>
    </operation>
    <operation name="obtenerPopulares">
        <soap:operation soapAction="obtenerPopulares"/>
        <input><soap:body use="literal"/></input>
        <output><soap:body use="literal"/></output>
    </operation>
</binding>

<service name="PalabrasService">
    <port name="PalabrasPort" binding="tns:PalabrasBinding">
        <soap:address location="http://localhost:8001/wsdl"/>
    </port>
</service>
</definitions>

```

SERVER JS

```

import express from 'express';
import fs from 'fs';
import soap from 'soap';
import path from 'path';
import { fileURLToPath } from 'url';
import { palabrasService } from './controllers/palabrasSoapController.js';

```

```
const __dirname = path.dirname(fileURLToPath(import.meta.url));
const app = express();
const port = 8001;

const wsdlXml = fs.readFileSync(path.join(__dirname, 'wsdl', 'palabras.wsdl'), 'utf8');

const server = app.listen(port, () => {
  console.log(`🚀 Microservicio SOAP corriendo en http://localhost:${port}/wsdl`);
});

soap.listen(server, '/wsdl', palabrasService, wsdlXml);
```

6. Conclusión

Este informe ha detallado el proceso completo de diseño, implementación, prueba y documentación de un microservicio cuyo objetivo principal es consumir un endpoint SOAP externo proporcionado por un proyecto integrador. Siguiendo los criterios de evaluación establecidos, se ha realizado un análisis exhaustivo del endpoint SOAP, identificando su estructura WSDL, operaciones y formato de datos.

El diseño del microservicio se ha basado en principios sólidos como la Responsabilidad Única, el Acoplamiento Débil y la Alta Cohesión, justificando su rol como fachada o adaptador entre el mundo SOAP externo y un ecosistema interno potencialmente basado en REST/JSON. Se ha detallado la implementación técnica utilizando Java, Spring Boot y JAX-WS, incluyendo la generación del cliente SOAP, el consumo de las operaciones externas, la transformación de datos necesaria y la exposición de una API REST propia y documentada mediante OpenAPI.

Se ha descrito una estrategia de pruebas comprensiva, abarcando pruebas unitarias con mocking de dependencias SOAP y pruebas de integración utilizando mocking a nivel HTTP para validar la correcta interacción con el contrato SOAP sin depender del servicio real. Se han delineado las mejores prácticas para la documentación, tanto a nivel de código (README.md) como a nivel técnico-arquitectónico (documento PDF).

Finalmente, se ha estructurado el contenido necesario para una presentación final, incluyendo la explicación técnica detallada y la demostración funcional clara del microservicio. El microservicio resultante cumple exitosamente su objetivo, encapsulando la complejidad de la

interacción SOAP y proporcionando una interfaz moderna y desacoplada para sus consumidores.

7. Referencias

La información y las directrices presentadas en este informe se basan en el análisis de las especificaciones proporcionadas y el material de investigación asociado, referenciado a lo largo del documento mediante identificadores como ¹, etc.

Obras citadas

1. Explaining WSDL Basics - SAP Learning, fecha de acceso: mayo 2, 2025, https://learning.sap.com/learning-journeys/developing-soap-web-services-on-sap-erp/explaining-wsdl-basics_c529eee3-752c-4a2e-a2d4-9974a1700b44
2. The simple guide to WSDL (with an example) - Tutorial Works, fecha de acceso: mayo 2, 2025, <https://www.tutorialworks.com/wsdl/>
3. WSDL Quick Guide - Tutorialspoint, fecha de acceso: mayo 2, 2025, https://www.tutorialspoint.com/wsdl/wsdl_quick_guide.htm
4. What is WSDL in SOAP | A Comprehensive Guide - DreamFactory Blog, fecha de acceso: mayo 2, 2025, <https://blog.dreamfactory.com/what-is-wsdl-in-soap-a-comprehensive-guide>
5. WSDL Introduction - GeeksforGeeks, fecha de acceso: mayo 2, 2025, <https://www.geeksforgeeks.org/wsdl-introduction/>
6. SOAP Message Structure - Tutorialspoint, fecha de acceso: mayo 2, 2025, https://www.tutorialspoint.com/soap/soap_message_structure.htm
7. Basics of SOAP – Simple Object Access Protocol | GeeksforGeeks, fecha de acceso: mayo 2, 2025, <https://www.geeksforgeeks.org/basics-of-soap-simple-object-access-protocol/>
8. Structure of a SOAP message - IBM, fecha de acceso: mayo 2, 2025, <https://www.ibm.com/docs/en/cics-ts/6.x?topic=format-structure-soap-message>
9. The structure of a SOAP message - IBM, fecha de acceso: mayo 2, 2025, <https://www.ibm.com/docs/en/integration-bus/10.0?topic=soap-structure-message>